3-board prototype wiring diagrams

Arduino Nano ESP32 - Star Destroyer 1/2700 lighting

What this is

The full 19-channel circuit laid out across your three 830-point breadboards, as a bench prototype before you commit to the soldered perfboard build. One SunFounder BreadVolt module (set to 5V) powers the whole thing. Boards 2 and 3 are then fed straight from Board 1 by two jumper wires each, so you only need a single USB power source for all three boards. The three boards clip together side by side along their long edges, which makes one square block. Put **Board 1 in the middle**, with Board 2 on its left and Board 3 on its right: both neighbours are then right beside the Nano, so the signal jumpers and the power jumpers are all short.

How the BreadVolt sits on Board 1

The BreadVolt is as wide as the breadboard. It lies across the top of Board 1 and has **four pins on each side**: a + and - pair at the front, and a + and - pair at the rear, which push into the + and - rail holes along both long edges of the board. It runs down the board as far as **row 5**, so it covers rows 1 to 5 completely and **nothing else can be plugged into those rows**. Everything else on Board 1 starts below it. Because it feeds the rails on both sides, all four rails of Board 1 are live as soon as it is plugged in - there are no wires from the module itself.

Board 1 rows	What goes there
1 - 5	BreadVolt power module (covers the full width of the board)
6 - 7	Left empty: clearance for the Nano's overhang. The power jumpers to Boards 2 and 3 plug into the rails here
8 - 22	Arduino Nano ESP32 - its two 15-pin rows straddle the centre gap, one pin per row, in columns **c** and **g**
23 - 24	Left empty
25 - 33	Driver C (TBD62083APG) - engines 1 to 7; pin 1 at row 25, pin 9 at row 33

How to read the breadboard drawings

The dots represent breadboard holes, and the small numbers down the left edge are the row numbers. On a real breadboard, every row of 5 holes on one side of the centre gap (columns **a-e**, or **f-j**) is one electrically connected node - so a chip pin plugged into column e of a row is the same connection as anything else plugged into columns a-d of that *same* row. Different rows are *not* connected to each other, which is why the Nano's pins need a jumper wire to reach Driver C, even though both sit on the same board. Each 18-pin driver chip straddles the gap so its left row of pins (1-9) sits in columns a-e and its right row of pins (18-10) sits in columns f-j, one pin per row, notch at the top. The Nano is wider than a driver chip - its two pin rows are 0.6 inch apart, not 0.3 - so its pins land in columns **c** and **g**, its body covers columns d, e and f, and the holes you can still use are a and b on the left and h, i and j on the right. The two long stripes down each edge are the **power rails**: every hole in a rail is connected all the way down the board. Each channel's LED sits lower down the board, in its own spot with **two empty holes** between it and the next LED, so the legs and fingers have room. An orange jumper runs from the driver's output to that LED, and the LED's resistor goes to the **left-hand + rail**. Coloured lines are jumper wires and can take any path on your boards. Clipping the boards together side by side (as sold) is purely mechanical - it does not connect their rails, which is why every link between boards is drawn as an explicit wire.

Rail order differs between breadboard brands, so always go by the + and - printed on your own board; the drawings show the left pair as + (outside) then -, and the right pair as + (inside) then -. Before connecting anything else, power the BreadVolt on its own and check with a multimeter that the red (+) rails read about 5 V

against the blue (-) rails. Also check that the rails run unbroken down the whole board (some split in the middle - bridge them if so).

Colour key

Colour	Meaning
Red	+5 V: the + rails, and wires carrying 5 V (including the jumpers between boards)
Black	GND (0 V): chip ground pins, Nano ground, and the jumper that carries GND to the next board
Blue / red rail stripes	The power rails: red = + (5 V), blue = - (0 V)
Blue numbered wires	Short jumpers on the same board - the same number at both ends is one wire (Nano D2-D8 to Driver C)
Violet wires	Longer jumpers that leave the board for another board; the label gives the destination
Orange	Jumper from a driver output pin to its LED (numbered tags show the two ends of each jumper); the LED then goes through its resistor to the left-hand + rail
LED symbol	The LED stands upright in column c across two rows: flat side / short leg (cathode) in the upper hole, where the orange jumper arrives; long leg (anode) in the hole below, which goes via the resistor to the + rail
Amber box	The BreadVolt power module (Board 1 only, rows 1-5)
Grey row label / "spare (open)"	Chip pin not used on this channel - leave it unconnected

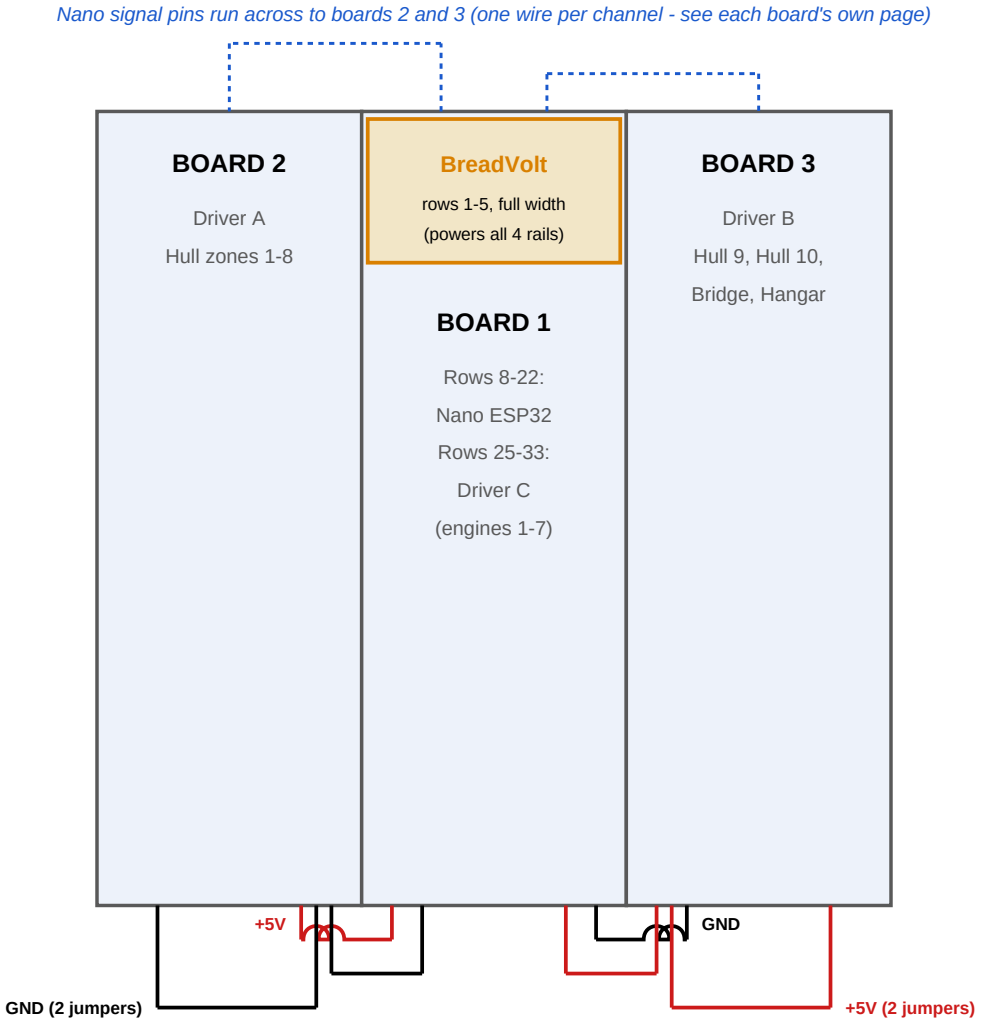
How to test it

Once everything below is wired up, use **pin_test_nano.ino** (in this same Testing document set) to check it, before you commit to soldering anything. This is the same sketch used again later in the permanent-build guide's Stage 5, to re-check the soldered boards - proving the design here means that later step only has to catch soldering mistakes, not design ones.

- **[ ]** Set the BreadVolt voltage switch to **5 V**, then plug it into the top of Board 1 across rows 1-5 with all eight pins in the rail holes. Power it over USB-C and check with a multimeter that the red rails read about 5 V against the blue rails on both sides. Unplug it before wiring.
- **[ ]** Wire up all three boards exactly as drawn over the following pages. Nothing goes in rows 1-5 of Board 1, and keep rows 6-7 clear of components.
- **[ ]** Upload **pin_test_nano.ino** to the Nano ESP32 over its USB-C data cable.
- **[ ]** Power the rails from the BreadVolt as shown (set to 5V) - the Nano is powered separately, by the same USB-C cable used to upload the sketch. The Nano's GND and the rails' GND are joined by the Nano GND jumper, so everything shares one 0 V.
- **[ ]** Confirm every channel lights in turn: engines 1 to 7, bridge, hangar bay, hull 1 to 10 - and that the engines fade individually rather than just switching on and off.
- **[ ]** Fix any channel that does not behave before moving on to the permanent build - it is much easier to debug here than after everything is soldered into the hull.

Layout overview

Board 1 carries the BreadVolt (rows 1-5), the Nano ESP32 plugged straight into the breadboard (rows 8-22) and Driver C (rows 25-33, all 7 engines); the engine LEDs sit lower down (rows 38-63), and each board's LEDs are spaced four rows apart. Boards 2 and 3 carry Driver A and Driver B and sit either side of it (Board 2 on the left, Board 3 on the right). They take their power from Board 1 over two jumpers each, and their signals over jumper wires from the Nano - neither needs a power module of its own, and neither is fed through the other.

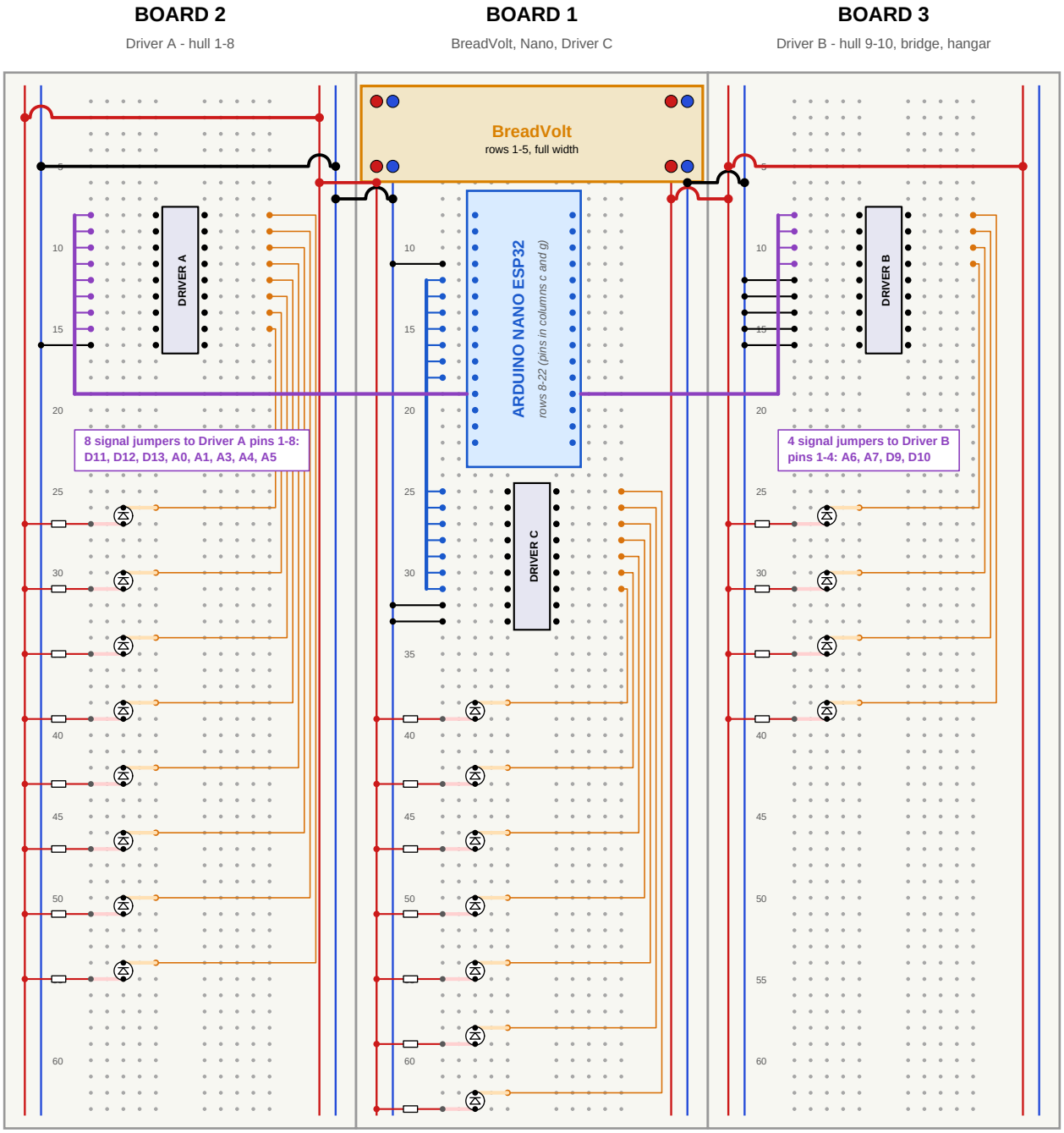


A link that has to cross a whole board is made as two jumpers, via the near rail of that board: a short one from Board 1, then a second from a different hole of that rail across the board (two wires cannot share a hole).

Only board 1 needs a power source (the BreadVolt module). Board 1 goes in the middle so both neighbours are right beside it.

All three boards side by side

The same layout drawn to scale, as it will sit on your bench: Board 2 on the left, **Board 1 in the middle**, Board 3 on the right. Three 830-point boards clipped along their long edges make a 165 x 165 mm square. Every hole is drawn at its real 2.54 mm spacing, so you can see how short the links between neighbouring boards are. The individual board pages that follow show the same wiring in readable detail.



To scale: three 830-point boards (55 x 165 mm each) clipped along their long edges make one 165 x 165 mm square. Board 1 is in the middle.

Board 1: 7 short blue jumpers, Nano D2-D8 -> Driver C pins 1-7. Rows 6-7 stay empty: the power jumpers to Boards 2 and 3 plug in there.
LEDs stand upright in column c across two rows, 4 rows apart (2 empty holes between). Board 2 gets one extra + jumper (row 2) to its left + rail.

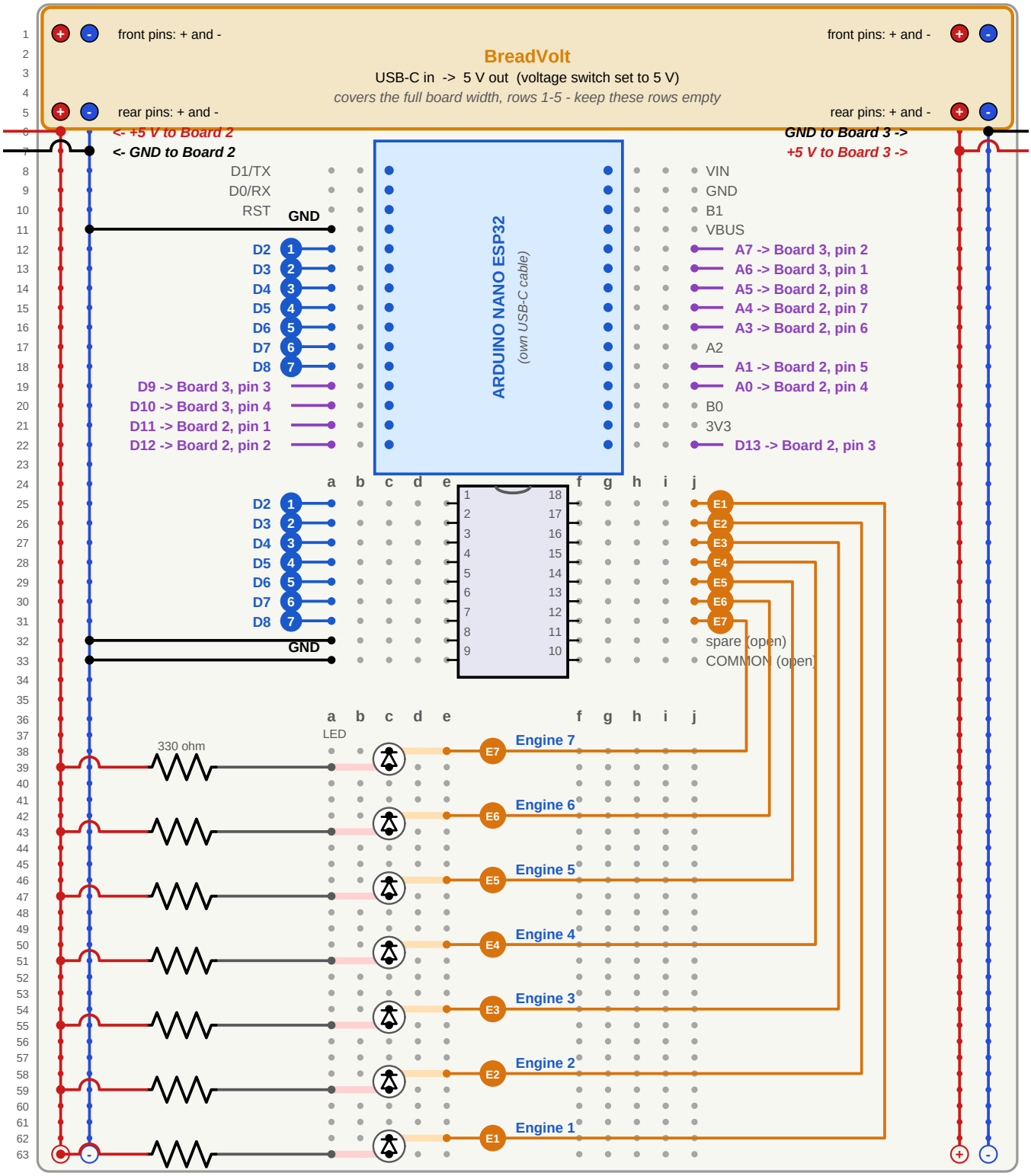
+5 V jumpers / + rails | GND jumpers | signal jumpers from the Nano | short jumpers on Board 1 | driver output -> LED (upright) -> resistor -> left + rail

Board 1 - BreadVolt, Nano ESP32 and Driver C

Top to bottom: the BreadVolt across rows 1-5 with its eight pins in the rail holes, the Nano ESP32 in rows 8-22 and Driver C in rows 25-33. The Nano and the driver are on different rows, so the seven engine signals D2-D8 need seven short jumper wires (blue, numbered 1-7). Both GNDs go to the left - rail. +5 V and GND then go to Board 2 (on the left) and Board 3 (on the right) from rail holes in rows 6 and 7, which are left empty for that.

BOARD 1 - BreadVolt, Arduino Nano ESP32, Driver C and the engine LEDs

rows 1-5: BreadVolt | rows 8-22: Nano ESP32 (USB-C end at the bottom) | rows 25-33: Driver C | rows 38-63: engine LEDs



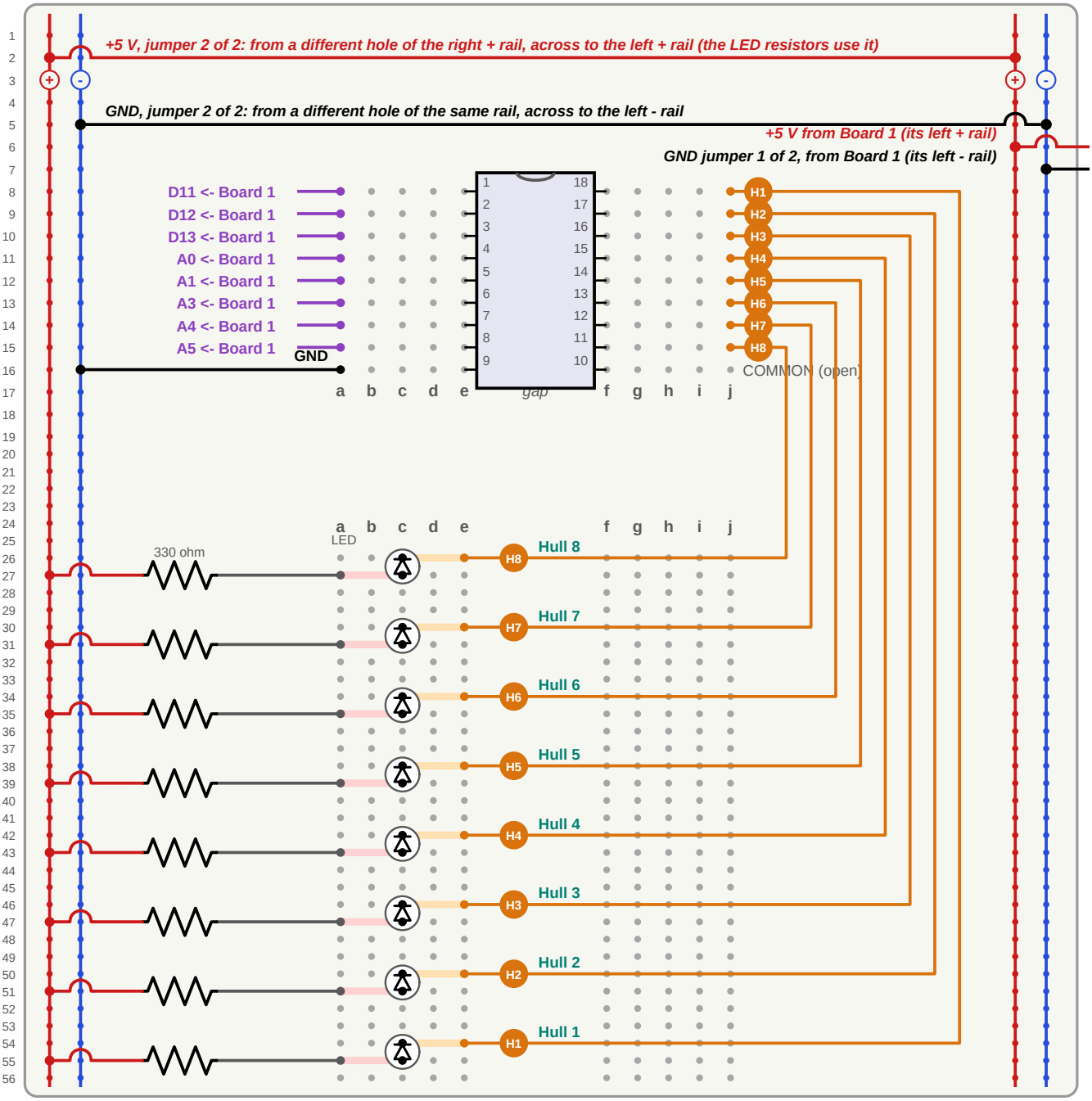
Blue numbered wires 1-7 are short jumpers on this board: Nano D2-D8 to Driver C pins 1-7. Orange wires E1-E7 run from each driver output to its LED (E1 = lowest LED). Each LED stands in column c across two rows (flat side / short leg up) with a 2-hole gap to the next. Violet wires leave for Board 2 / Board 3. The Nano has its own USB-C cable. Nano GND (row 11) and Driver C GND (rows 32-33) go to the left - rail so everything shares one 0 V.

Board 2 - Driver A (hull zones 1-8)

All 8 of Driver A's channels are used here, one per hull zone. Board 2 sits on the left of Board 1. Power arrives from Board 1's left-hand rails: +5 V in one jumper (row 6, to the right + rail) and GND in two, plus a third wire from the right + rail to this board's left + rail (row 2, the LED resistors use it); because one wire cannot cross the whole board and two wires cannot share a hole - a short jumper from Board 1 into this board's right-hand - rail (row 7), then a second from a different hole of that rail (row 5) across to the left - rail; the Nano's signal pins (D11, D12, D13, A0, A1, A3, A4, A5) run across from Board 1 as jumpers.

BOARD 2 - Driver A (hull zones 1-8)

all 8 channels used - hull zones 1-8



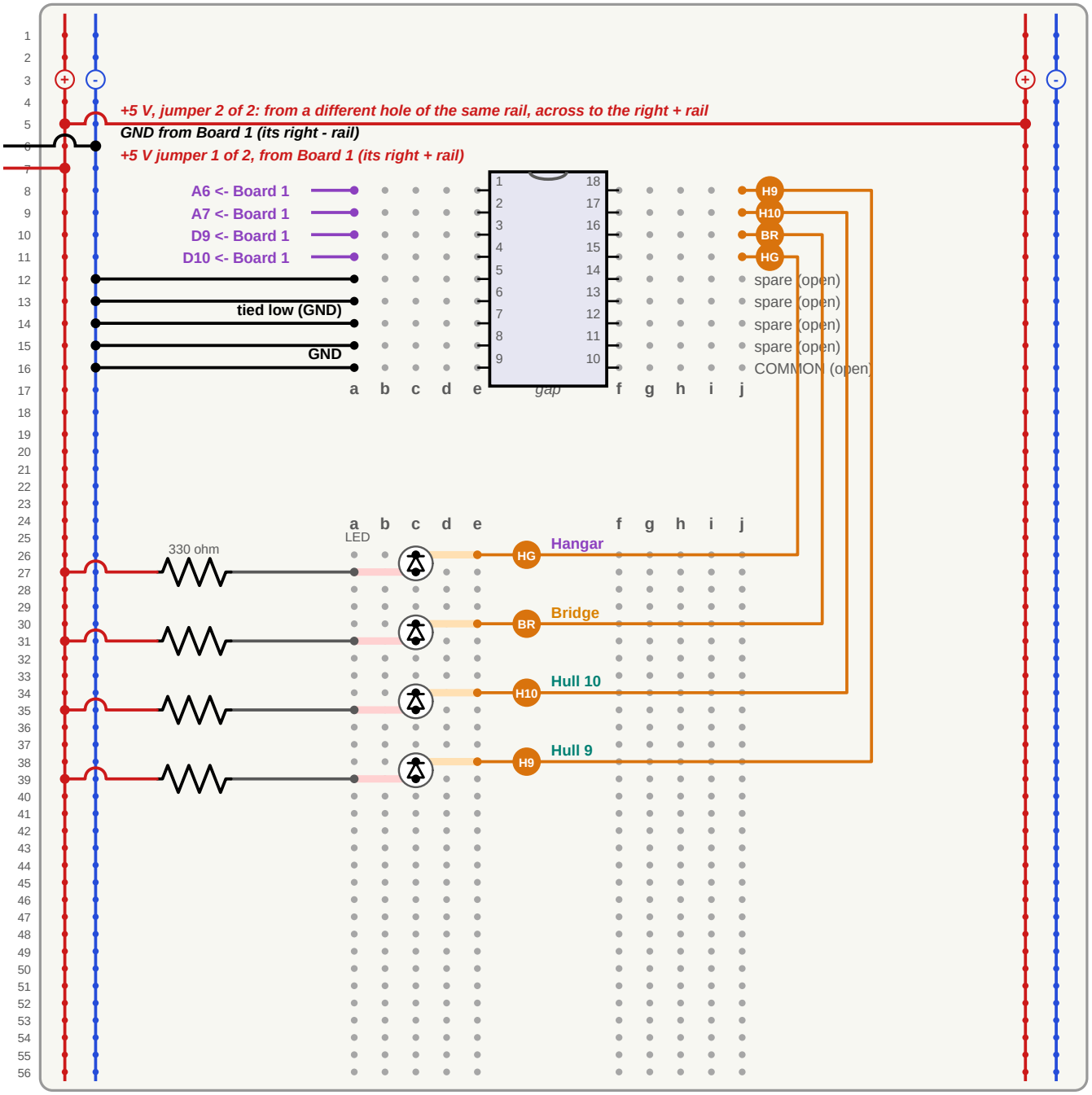
Each channel is drawn with one LED and resistor, 4 rows apart (2 empty holes between LEDs). In the permanent build each zone is a cluster of 4 LEDs in parallel (each with its own resistor) - one LED per channel is enough to prove the wiring here.
No power module on this board: fed from Board 1 (on its right): +5 V in one jumper, GND in two, plus the extra + jumper to the left + rail (see above).

Board 3 - Driver B (hull 9-10, bridge, hangar)

Only 4 of Driver B's 8 channels are used (Hull 9, Hull 10, Bridge, Hangar); the other 4 inputs are tied low and their outputs left open, the same way Driver C's spare channel is handled on Board 1. Board 3 sits on the right of Board 1, and its power comes from Board 1's right-hand rails, not through Board 2: GND in one jumper (row 6, to the left - rail) and +5 V in two - a short jumper from Board 1 into this board's left-hand + rail (row 7), then a second from a different hole of that rail (row 5) across to the right + rail.

BOARD 3 - Driver B (hull 9-10, bridge, hangar)

4 of 8 channels used - hull 9, hull 10, bridge, hangar



Each channel is drawn with one LED and resistor, 4 rows apart (2 empty holes between LEDs). In the permanent build each zone is a cluster of 4 LEDs in parallel (each with its own resistor) - one LED per channel is enough to prove the wiring here.
Fed from Board 1 (on its left): GND in one jumper, +5 V in two (see above).

Appendix - Pin tester sketch (pin_test_nano.ino)

Upload this to the Nano ESP32 to run the test on the previous page. It lights every channel in turn (engines 1-7, bridge, hangar bay, hull 1-10), then runs the engines through their PWM fade so you can confirm that behaves correctly too. It is the same sketch referenced again in Stage 5 of the permanent build guide, used there to re-check the soldered boards.

```
// PIN TESTER - ARDUINO NANO ESP32
// Part 1: lights EVERY one of the 19 channels in turn, in order:
//   engines 1-7, bridge, hangar bay, hull zones 1-10.
//   This alone confirms every wire and every LED cluster is connected correctly.
// Part 2: additionally fades the 7 engines up and down one after another (PWM
//   check) - engines are the only channels that use PWM, so this is an extra
//   check on top of Part 1, not a replacement for it.
// Repeats forever.
//
// Engine PWM uses Espressif's LEDC calls (core 2.x: ledcSetup/ledcAttachPin/ledcWrite(channel),
// core 3.x: ledcAttach/ledcWrite(pin)); the right ones are chosen automatically.

#if __has_include("esp_arduino_version.h")
#include "esp_arduino_version.h"
#endif
#if defined(ESP_ARDUINO_VERSION_MAJOR) && (ESP_ARDUINO_VERSION_MAJOR >= 3)
#define PWM_CORE_V3 1
#else
#define PWM_CORE_V3 0
#endif

const uint32_t PWM_FREQ = 5000;
const uint8_t PWM_BITS = 12;
const uint32_t PWM_MAX = (1UL << PWM_BITS) - 1;

const uint8_t N_ENG = 7;
const uint8_t ENG_PINS[N_ENG] = {D2, D3, D4, D5, D6, D7, D8};
const uint8_t ZONE_PINS[] = {D9, D10,                                   // bridge, hangar
                             D11, D12, D13, A0, A1, A3, A4, A5, A6, A7}; // hull 1-10
const uint8_t N_ZONE = sizeof(ZONE_PINS);

void setEngine(uint8_t i, uint32_t duty) {
  if (duty > PWM_MAX) duty = PWM_MAX;
#if PWM_CORE_V3
  ledcWrite(ENG_PINS[i], duty);
#else
  ledcWrite(i, duty);
#endif
}

void setup() {
  for (uint8_t i = 0; i < N_ENG; i++) {
    pinMode(ENG_PINS[i], OUTPUT);
#if PWM_CORE_V3
    ledcAttach(ENG_PINS[i], PWM_FREQ, PWM_BITS);
#else
    ledcSetup(i, PWM_FREQ, PWM_BITS);
    ledcAttachPin(ENG_PINS[i], i);
#endif
    setEngine(i, 0);
  }
  for (uint8_t i = 0; i < N_ZONE; i++) { pinMode(ZONE_PINS[i], OUTPUT); digitalWrite(ZONE_PINS[i], LOW); }
}

void loop() {
  // Part 1: one at a time
  for (uint8_t i = 0; i < N_ENG; i++) {
    setEngine(i, PWM_MAX); delay(1000);
    setEngine(i, 0);       delay(300);
  }
  for (uint8_t i = 0; i < N_ZONE; i++) {
    digitalWrite(ZONE_PINS[i], HIGH); delay(1000);
    digitalWrite(ZONE_PINS[i], LOW);  delay(300);
  }
  // Part 2: PWM fade check on each engine
  for (uint8_t i = 0; i < N_ENG; i++) {
    for (int v = 0; v <= 255; v += 5) { setEngine(i, (uint32_t)v * PWM_MAX / 255); delay(15); }
    for (int v = 255; v >= 0; v -= 5) { setEngine(i, (uint32_t)v * PWM_MAX / 255); delay(15); }
  }
  delay(500);
}
```